

UNIVERSITATEA DIN BUCUREȘTI

Facultatea de Matematică și Informatică

PROIECT

**Detectia Conturilor în Imagini Digitale folosind
Operatori de Gradient**

Materie: Procesarea Semnalelor

Student: Vasile Radu-Mihai

Februarie 2026

Cuprins

Rezumat	2
1 Introducere	3
2 Fundamente Teoretice	4
2.1 Imaginea ca Semnal Discret	4
2.2 Gradientul și Derivata	4
3 Metodologie	5
3.1 Convoluția Discretă 2D	5
3.2 Operatorii Sobel	5
3.3 Magnitudinea Gradientului	5
3.4 Binarizare și Praguire (Thresholding)	6
4 Detalii de Implementare	7
5 Rezultate Experimentale	8
5.1 Analiza pentru Prag Scăzut ($T=0.1$)	8
5.2 Analiza pentru Prag Moderat-Scăzut ($T=0.2$)	9
5.3 Analiza pentru Prag Mediu ($T=0.3$)	10
5.4 Analiza pentru Prag Ridicat ($T=0.4$)	11
5.5 Analiza pentru Prag Foarte Ridicat ($T=0.5$)	12
6 Concluzii	13

Rezumat

Acest proiect prezintă implementarea detaliată și analiza critică a unui algoritm fundamental în domeniul procesării imaginilor: detecția contururilor. Obiectivul principal al lucrării este extragerea informației structurale dintr-o imagine digitală prin identificarea schimbărilor bruște de intensitate luminoasă.

Metodologia utilizată se bazează pe filtrarea spațială prin convoluție discretă 2D, utilizând operatorii de gradient Sobel, o tehnică clasică dar robustă în viziunea artificială. Procesul a fost implementat integral în limbajul Python, evitând utilizarea funcțiilor de nivel înalt (black-box) pentru detecție automată, cu scopul de a demonstra o înțelegere profundă a fenomenului matematic.

Rezultatele experimentale ilustrează transformarea imaginii originale într-o hartă de gradient și, ulterior, într-o hartă binară de contur prin operația de praguire (thresholding). Lucrarea include o analiză extensivă a sensibilității algoritmului față de parametrul de prag, demonstrând impactul acestuia asupra calității finale a contururilor extrase.

1 Introducere

În era digitală actuală, imaginile reprezintă una dintre cele mai bogate surse de informație. De la diagnosticarea medicală prin RMN și până la vehiculele autonome care navighează în trafic, capacitatea unui computer de a "înțelege" o imagine este crucială. Totuși, pentru un calculator, o imagine nu este altceva decât o matrice imensă de numere.

În domeniul procesării semnalelor, o imagine digitală poate fi considerată un semnal bidimensional, $f[x, y]$, unde valoarea funcției într-un punct reprezintă intensitatea pixelului. Una dintre cele mai importante etape în analiza imaginilor și în viziunea artificială (Computer Vision) este simplificarea reprezentării imaginii prin păstrarea doar a informațiilor esențiale despre formă și structură, eliminând detaliile redundante.

Detecția contururilor (Edge Detection) rezolvă această problemă prin identificarea pixelilor unde luminozitatea se schimbă drastic. Aceste discontinuități corespund, de regulă, proprietăților fizice importante ale scenei:

- Granițele dintre obiecte distincte;
- Liniile de demarcație dintre lumini și umbre;
- Schimbările de orientare a suprafețelor;
- Variațiile de textură sau material.

Relevanța acestei tehnici este majoră. Fără o detecție corectă a contururilor, algoritmi de nivel superior (cum ar fi recunoașterea facială sau interpretarea scenelor) ar eșua. Prezentul proiect urmărește implementarea matematică a detecției de contur folosind aproximarea derivatei discrete a semnalului imagine, oferind o perspectivă "sub capotă" a modului în care mașinile percep formele.

2 Fundamente Teoretice

Pentru a înțelege metodologia aplicată, este necesar să definim cadrul matematic al imaginilor digitale și conceptul de derivată în spațiul discret.

2.1 Imaginea ca Semnal Discret

O imagine digitală I de dimensiune $M \times N$ este definită ca o funcție discretă de două variabile, $I(x, y)$, unde x și y sunt coordonatele spațiale, iar valoarea funcției reprezintă intensitatea. Într-o imagine "grayscale" (tonuri de gri) pe 8 biți, intensitatea variază în intervalul $[0, 255]$, unde 0 reprezintă negru absolut, iar 255 reprezintă alb absolut.

Pentru procesare matematică precisă, în acest proiect am normalizat imaginile în intervalul $[0.0, 1.0]$, transformându-le în format `float`.

2.2 Gradientul și Derivata

Conturul este, prin definiție, o zonă de schimbare rapidă a intensității. În calculul diferențial continuu, rata de schimbare este dată de derivată. Pentru o funcție 2D $f(x, y)$, gradientul este un vector definit astfel:

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \quad (1)$$

Magnitudinea acestui vector ne spune cât de repede se schimbă imaginea (cât de "puternic" este conturul), iar direcția vectorului ne spune orientarea conturului. Deoarece imaginile sunt discrete (formate din pixeli), nu putem calcula derivate infinitesimale. Trebuie să le aproximăm folosind ****diferențe finite****. Cea mai simplă aproximare a derivatei pe axa x este:

$$\frac{\partial f}{\partial x} \approx f(x + 1, y) - f(x - 1, y)$$

Această operație stă la baza filtrelor de detecție a marginilor.

3 Metodologie

Abordarea tehnică a acestui proiect se bazează pe teoria filtrării liniare. Procesul complet constă în trei etape majore: convoluția cu filtre de derivare, calculul magnitudinii gradientului și binarizarea finală.

3.1 Convoluția Discretă 2D

Convoluția este operația fundamentală în procesarea imaginilor. Ea presupune "glisarea" unei matrici mici (numită nucleu, filtru sau kernel) peste întreaga imagine. Matematic, pentru un pixel (x, y) , convoluția dintre imaginea I și nucleul K este:

$$(I * K)[x, y] = \sum_{i=-1}^1 \sum_{j=-1}^1 I[x - i, y - j] \cdot K[i, j] \quad (2)$$

Această sumă ponderată permite fiecărui pixel să fie recalculat în funcție de vecinii săi.

3.2 Operatorii Sobel

Pentru a detecta marginile, am utilizat operatorii Sobel. Aceștia sunt preferați operatorilor simpli de diferențiere deoarece includ o componentă de netezire (blurring) pe direcția perpendiculară derivatei. Acest lucru face algoritmul mai rezistent la zgomotul de imagine ("purici").

Nucleele de convoluție utilizate sunt:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad \text{și} \quad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \quad (3)$$

Operatorul G_x răspunde maxim la linii verticale (calculând diferența dintre dreapta și stânga), iar G_y răspunde la linii orizontale (calculând diferența dintre sus și jos).

3.3 Magnitudinea Gradientului

După aplicarea convoluției, obținem două imagini intermediare: E_x (contururi verticale) și E_y (contururi orizontale). Pentru a obține conturul final, combinăm cele două componente folosind norma Euclidiană (teorema lui Pitagora):

$$M[x, y] = \sqrt{G_x[x, y]^2 + G_y[x, y]^2} \quad (4)$$

3.4 Binarizare și Praguire (Thresholding)

Rezultatul M este o imagine în tonuri de gri. Pentru a lua o decizie finală (este sau nu este margine?), aplicăm un prag T .

$$C[x, y] = \begin{cases} 1(Alb), & \text{dacă } M[x, y] > T \\ 0(Negru), & \text{altfel} \end{cases} \quad (5)$$

4 Detalii de Implementare

Proiectul a fost realizat în limbajul **Python**, ales datorită ecosistemului său vast pentru calcul științific.

Bibliotecile principale utilizate sunt:

- **NumPy:** Folosită pentru manipularea eficientă a matricelor și a operațiilor vectorizate. Imaginile au fost stocate ca `numpy.array`.
- **SciPy (Signal):** S-a utilizat funcția `scipy.signal.convolve2d` pentru a implementa formula matematică a convoluției 2D. S-a folosit parametrul `boundary='symm'` pentru a trata corect pixelii de la marginea imaginii prin oglindire.
- **Matplotlib:** Utilizată pentru vizualizarea rezultatelor și generarea graficelor comparative.
- **Skimage (Scikit-Image):** Utilizată strict pentru încărcarea imaginii și conversia acesteia din format RGB în Grayscale.

Codul a fost structurat modular, permițând ajustarea ușoară a parametrului de prag (threshold) pentru a testa sensibilitatea algoritmului, așa cum va fi prezentat în secțiunea următoare.

5 Rezultate Experimentale

Un aspect critic al detecției de contur este alegerea pragului de binarizare (T). Acest parametru dictează sensibilitatea algoritmului: cât de "puternică" trebuie să fie o schimbare de culoare pentru a fi considerată muchie.

În continuare, prezentăm rezultatele obținute pe o imagine de test, variind pragul de la 0.1 la 0.5. Imaginile sunt prezentate individual pentru a permite o analiză detaliată a texturii și a zgomotului.

5.1 Analiza pentru Prag Scăzut ($T=0.1$)

La un prag foarte scăzut, algoritmul este extrem de permisiv.

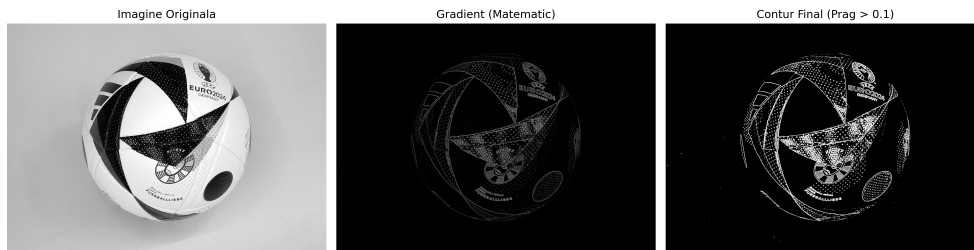


Figura 1: Rezultat pentru Prag $T = 0.1$. Se observă o densitate foarte mare a conturilor.

După cum se observă în Figura 1, conturile sunt foarte groase. Mai mult, algoritmul a detectat o cantitate semnificativă de "zgomot" – mici variații de textură care nu reprezintă contururi reale ale obiectelor, ci doar detalii irelevante sau imperfecțiuni ale imaginii.

5.2 Analiza pentru Prag Moderat-Scăzut ($T=0.2$)

Creșterea ușoară a pragului începe să filtreze zgomotul de fond.

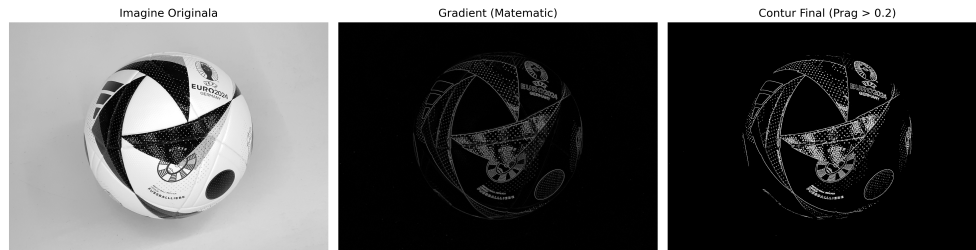


Figura 2: Rezultat pentru Prag $T = 0.2$. Zgomotul începe să dispară.

În Figura 2, imaginea devine mai curată. Contururile principale sunt bine definite și continue. Acesta este adesea un punct de plecare bun pentru multe aplicații de viziune artificială, deoarece păstrează majoritatea detaliilor structurale fără a fi copleșit de informație falsă.

5.3 Analiza pentru Prag Mediu ($T=0.3$)

Acesta reprezintă adesea echilibrul optim pentru imagini cu contrast normal.

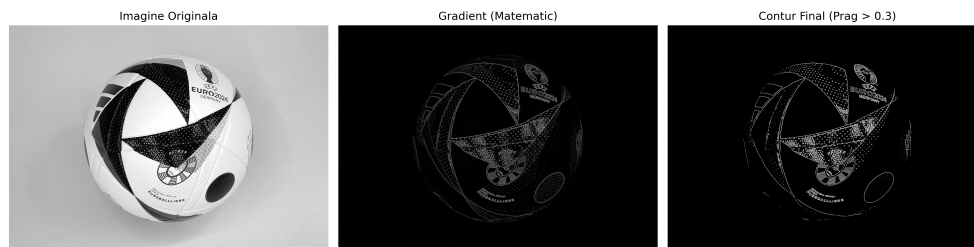


Figura 3: Rezultat pentru Prag $T = 0.3$. Contururi clare și fundal curat.

Figura 3 demonstrează o separare foarte bună a obiectului de fundal. Liniile sunt mai subțiri decât în cazurile anterioare, ceea ce oferă o precizie mai bună a localizării spațiale a muchiilor. Zgomotul este aproape inexistent.

5.4 Analiza pentru Prag Ridicat ($T=0.4$)

Pe măsură ce creștem pragul, algoritmul devine foarte selectiv, păstrând doar muchiile cu contrast foarte puternic.

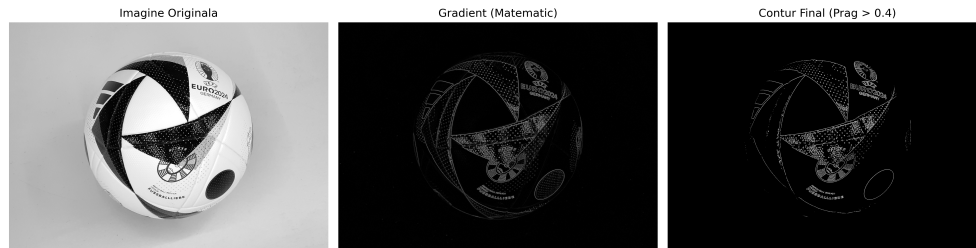


Figura 4: Rezultat pentru Prag $T = 0.4$. Începe pierderea detaliilor fine.

În Figura 4, se observă că unele detalii mai fine încep să dispară. Contururile pot apărea ușor întrerupte în zonele unde iluminarea nu este ideală sau unde contrastul obiect-fundal este scăzut.

5.5 Analiza pentru Prag Foarte Ridicat ($T=0.5$)

La acest nivel, doar cele mai evidente trăsături ale imaginii sunt păstrate.

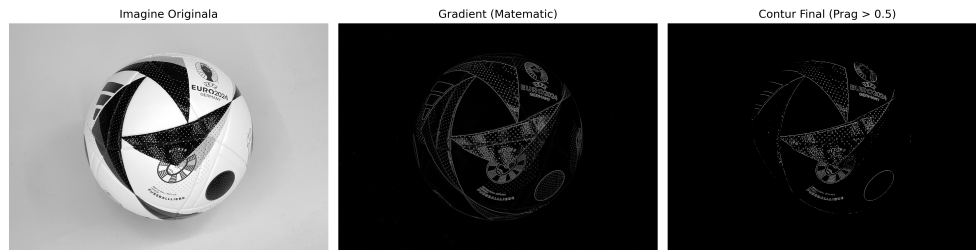


Figura 5: Rezultat pentru Prag $T = 0.5$. Contururi fragmentate.

Figura 5 arată efectul unei praguiri prea agresive. Multe contururi utile au fost eliminate ("șterse"), iar ceea ce a rămas sunt doar segmente disperate. Deși imaginea este foarte curată (fără zgomot), pierderea informației structurale este prea mare pentru a fi utilă în recunoașterea obiectelor.

6 Concluzii

În cadrul acestui proiect, am demonstrat aplicabilitatea conceptelor teoretice de procesare a semnalelor în analiza imaginilor. Prin implementarea manuală a convoluției cu operatori Sobel, am evidențiat relația directă dintre derivata matematică a unui semnal și percepția vizuală a contururilor.

Experimentele efectuate și prezentate în paginile anterioare au condus la următoarele concluzii principale:

1. ****Operatorii Sobel sunt eficienți:**** Deși simpli din punct de vedere computațional, aceștia reușesc să extragă cu succes structura principală a unei imagini.
2. ****Importanța Pragului:**** Nu există o valoare "universală" pentru pragul de binarizare (T). O valoare prea mică (0.1) introduce zgomot, în timp ce o valoare prea mare (0.5) distruge conectivitatea muchiilor. Pentru imaginea testată, intervalul optim s-a dovedit a fi $0.2 - 0.3$.
3. ****Natura discretă:**** Procesarea imaginilor depinde critic de rezoluție și de aproximarea corectă a derivatelor.

Ca direcții viitoare de dezvoltare, algoritmul poate fi îmbunătățit prin:

- Adăugarea unui pas de pre-procesare cu ****filtru Gaussian**** mai puternic pentru reducerea zgomotului înainte de derivare.
- Implementarea metodei ****Canny****, care folosește histerezis (două praguri) pentru a conecta muchiile întrerupte, rezolvând problema observată la $T = 0.5$.
- Utilizarea transformatei Hough pentru a detecta forme geometrice (linii, cercuri) pe baza contururilor extrase.

Bibliografie

- [1] R. C. Gonzalez și R. E. Woods, *Digital Image Processing*, 4th Edition, Pearson, 2018.
- [2] C. Rusu, *Cursuri de Procesarea Semnalelor*, Facultatea de Matematică și Informatică, Universitatea din București, 2026.
- [3] I. Sobel și G. Feldman, "A 3x3 Isotropic Gradient Operator for Image Processing", *Stanford Artificial Intelligence Project*, 1968.

- [4] P. Virtanen et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python", *Nature Methods*, 2020.